Project Report

On

Prediction of Model Accuracy Degradation Using Statistical Drift Metrics

Submitted to Manipal University Jaipur

Towards the partial fulfillment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Computers Science and Engineering

2025-2026

By

Anwesha Singh

23FE10CSE00299

MANIPAL UNIVERSITY
JAIPUR

Signature of Supervisor

Department of Computer Science and Engineering

School of Computer Science and Engineering

Manipal University Jaipur

Jaipur, Rajasthan - 303007

ABSTRACT

This report discusses an end-to-end Machine learning pipeline developed to detect and predict data drift in deployed models. The work uses CICIDS 2017 dataset which has records of 5 full working days of real network traffic and has around 2.8 million records. The processing is done in 4 phases, the first being the preprocessing, baseline model training, statistical drift computation, and drift prediction modelling.

The core focus of this project is to use distributional divergence metrics like: Kullback-Leibler (KL) Divergence, Jensen-Shannon (JS) Divergence, and Wasserstein Distance, and calculate them using unlabelled feature distributions that are incoming. This helps predict how much a model's accuracy will degrade before a single inference is made. A Linear Regression meta-model trained on seven temporal data points achieves an R-squared value of 0.9924 and a Mean Absolute Error (MAE) of 1.43% using leave one out cross validation. The five baseline classifiers namely Random Forest, XGBoost, Decision Tree, Logistic Regression, and SVM, have all been observed to achieve the accuracy off more than 93% on the same distribution evaluation. An interesting observation was when Random Forest model trained on Monday is evaluated on Friday's DDoS traffic without retraining the accuracy severely degrades to 43.33%. This severe degradation was observed to be very well predicted by KL Divergence of 12.3652 between the reference and drifted distributions.

The whole pipeline is fully operational, deterministic, and memory safe. All outputs are saved in versioned artifact files, leading to reproducibility.

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Silent Model Decay

A lot of research reach some accuracy threshold on test set, and then they often end there. But in real world the application of model the actual work just starts there. Most models are often deployed with an assumption that the future data will behave in the similar manner as the training data which proves to be completely wrong in the real world.

Talking about network traffic, which in today's world the most dynamically shifting data any researcher or organization can encounter. The network traffic on a Monday morning is completely different from that on a Wednesday afternoon's DoS flood and a model that was trained on Monday will for sure show less accuracy on Wednesday data. This degradation when model encounters shifted data is known as silent model decay. It is called Silent Model Decay because there is often no sign of anything that has gone wrong during the operation, but the accuracy keeps degrading. Security Organizations may take days or even weeks to identify this before retraining and updating the accuracy metrics. By the time this degrade, and performance is noticed the damage to the network may have already occurred.

This project aims to target this problem. Instead of waiting for labelled data to confirm that the model's accuracy is degrading, the aim is to determine if the statistical properties of any incoming data could be used to predict the extent of expected performance degradation. The answer that was achieved from the results of this research indicates positive results for the same, at least for the CICIDS 2017 dataset.

2. PROBLEM STATEMENT

How can we detect data drift and accurately predict the expected degradation in model's accuracy using only statistical divergence metrics calculated on incoming feature distributions, eliminating the need of actual ground truth labels at the time of inference?

The significance of this problem cannot be emphasised enough in the cybersecurity world. If model can compute KL divergence between a stored reference distribution an incoming data batch within seconds, and if a pre-trained regression model can translate these divergence values into predicted accuracy numbers with below 2% error, then the deployed system could have everything it needs to alert the operators in advance before inference even begins. This could be critical for ML-based systems working on intrusion detection.

3. LITERATURE REVIEW

3.1 Reactive Drift Detection Methods

The most developed area of drift detection research focuses on retrospective monitoring, which means detecting drift that has already happened. ADWIN [4], proposed by Bifet and Gavaldà in 2007, maintains an adaptive sliding window over a monitored data stream. It detects drift by comparing the statistics of sub-windows when the difference between them exceeds a limit based on the Hoeffding inequality. This algorithm is sound in theory and efficient in practice, which explains its popularity in streaming frameworks.

3.3 Statistical Divergence as a Drift Proxy

The problem with KL is fixed by Jensen-Shannon (JS) Divergence. Finding the average KL from each distribution to their mixture shows that divergence is not symmetrical. JS Divergence is a lot more stable numerically and works better for threshold-based alerts than raw KL because it only works with numbers between zero and one. Rabanser et al. (2019) performed an empirical evaluation of multiple distributional drift detection techniques, demonstrating that JS-based methods are robust across various dataset categories and drift magnitudes. [5]

Wasserstein Distance, or the Earth Mover's Distance, is a geometrically simple way to measure how a distribution has changed. Instead of measuring information-theoretic divergence, it finds the least expensive way to change one distribution into another using optimal transport. This property makes it very sensitive to changes in the overall shape and location of a distribution, as opposed to pointwise probability differences. This is why it was so useful in our rolling window analysis, where overnight traffic changes caused Wasserstein spikes that were ten times bigger than the corresponding KL Divergence changes.

3.4 CICIDS 2017 Prior Works [1]

The CICIDS 2017 dataset was developed by Sharafaldin, Lashkari, and Ghorbani (2018) to overcome the flaws associated with older datasets like KDD99 and NSL-KDD, which have been extensively criticized for being overly simplistic, outdated, and non-representative of contemporary attacks. The CICFlowMeter utility was employed to derive 83 statistical characteristics related to bidirectional flow behavior from actual packet traces, which include flow duration, packet length distribution, inter-arrival time distribution, flag count, and window size. According to Ahmad et al. (2018), Random Forest attains almost perfect classification performance with respect to the CICIDS 2017 benchmark when the training and testing procedures are conducted on the same day, a result that has been confirmed independently by the authors with a performance score of 99.76%. However, no existing publication, to the best of the authors' knowledge, has attempted to examine the cross-day generalization problem: namely, how would the accuracy of a Monday-trained classifier be impacted by evaluating its performance on Friday's DDoS data without further training?

4. OBJECTIVES

- **Baseline model training:** Train baseline models whose accuracy degrade will be predicted.
- **Quantify the data drift:** Calculate and quantify drifts using metrics like, KL Divergence, Jensen-Shannon Divergence, and Wasserstein Distance, at both the label level and the per-feature level.
- **Cross dataset evaluation:** Demonstrate the relationship between distributional divergence and model accuracy degradation.
- **Data drift prediction Meta-model**: A regression based meta-model that can map divergence scores to accuracy.
- Save all results and observations for reproducibility

5. DATASET DESCRIPTION

CICIDS 2017 [1][3]

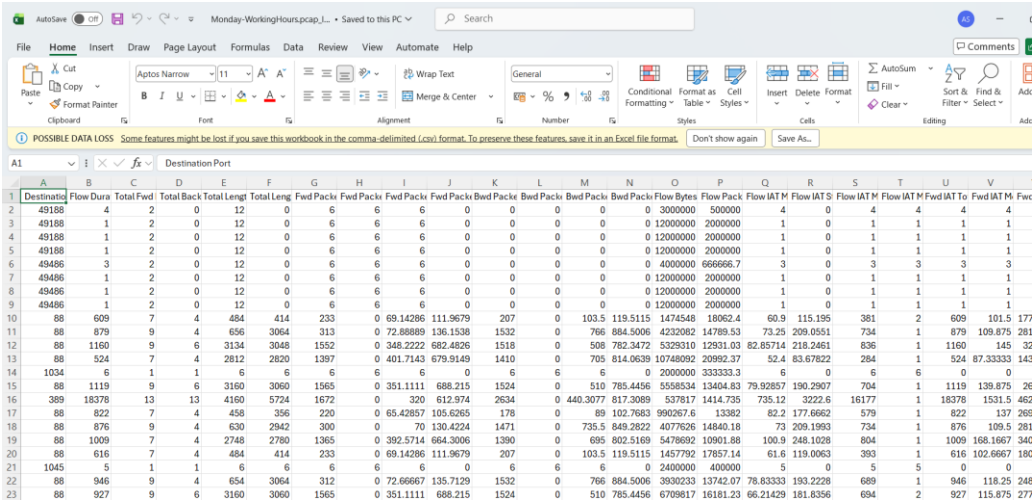
The Canadian Institute of Cybersecurity Intrusion Detection System 2017 (CICIDS 2017) dataset was chosen for this project because it satisfied two major requirements: the first one being that it is a realistic, and the second being that it is temporally dependent. Synthetic datasets or heavily processed academic datasets do not capture the real network traffic. This dataset not only captures the real network traffic but also provides authentic conditions over 5 working days. CICFlowMeter was used to extract 83 bidirectional statistical flow features from the raw PCAP files, which provides a rich set of characteristics including flow distribution, packet length statistics, inter-arrival time distributions, TCP flag counts and TCP window sizes.

The temporal day-by-day structure of this data set plays an important role in this project. Each day represents a different temporal window with a different attack profile

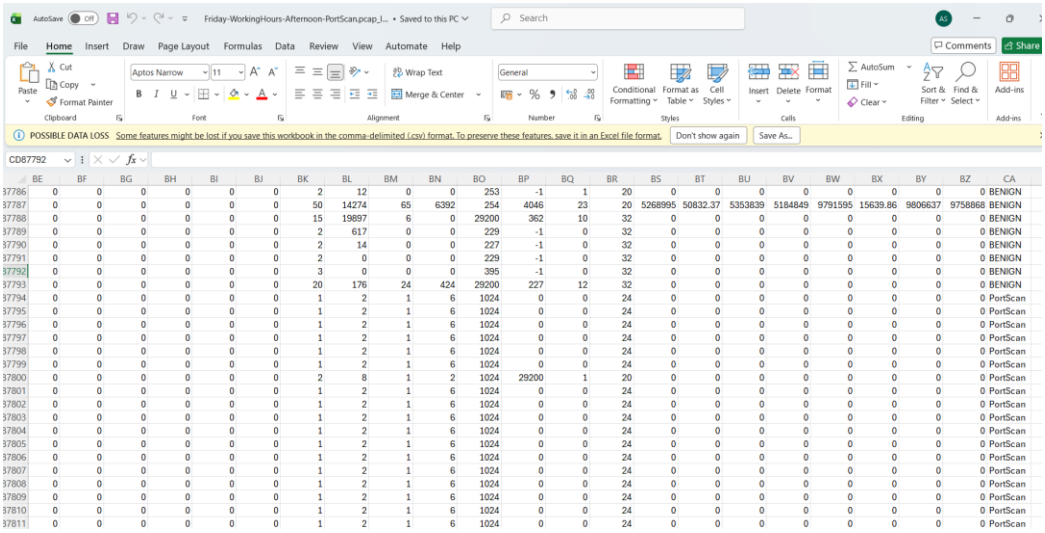
The data set is separated into 8 separate CSV files:

CSV	**Rows (PostProcessing)**	**NaN/Inf Removed**	**Primary Traffic Type**
Monday-WorkingHours	529,481	437	Benign (Reference)
Tuesday-WorkingHours	445,645	264	Brute Force (FTP, SSH)
Wednesday-WorkingHours	691,406	1,297	DoS
Thursday-Morning (Web Attacks)	170,231	135	SQL Injection, XSS
Thursday-Afternoon (Infiltration)	288,395	207	Infiltration
Friday-Morning	190,911	122	Port Scan
Friday-Afternoon (PortScan)	286,096	371	Port Scan
Friday-Afternoon (DDoS)	225,711	34	DDoS Attack
TOTAL	2,827876	2,867	

Table 1: CICIDS 2017 dataset structure with row counts from preprocessing logs



	Destinatio	Flow Dura	Total Fwd	Total Back	Total Lengt	Total Leng	Fwd Packe	Fwd Packe	Fwd Packe	Fwd Packe	Bwd Packe	Bwd Packe	Bwd Packe	Bwd Packe	Flow Bytes	Flow Pack	Flow IAT M	Flow IAT S	Flow IAT M	Flow IAT M	Fwd IAT To	Fwd IAT M	Fwc
2	49188	4	2	0	12	0	6	6	6	0	0	0	0	0	3000000	500000	4	0	4	4	4	4	
3	49188	1	2	0	12	0	6	6	6	0	0	0	0	0	12000000	2000000	1	0	1	1	1	1	
4	49188	1	2	0	12	0	6	6	6	0	0	0	0	0	12000000	2000000	1	0	1	1	1	1	
5	49188	1	2	0	12	0	6	6	6	0	0	0	0	0	12000000	2000000	1	0	1	1	1	1	
6	49486	3	2	0	12	0	6	6	6	0	0	0	0	0	4000000	666666.7	3	0	3	3	3	3	
7	49486	1	2	0	12	0	6	6	6	0	0	0	0	0	12000000	2000000	1	0	1	1	1	1	
8	49486	1	2	0	12	0	6	6	6	0	0	0	0	0	12000000	2000000	1	0	1	1	1	1	
9	49486	1	2	0	12	0	6	6	6	0	0	0	0	0	12000000	2000000	1	0	1	1	1	1	
10	88	609	7	4	484	414	233	0	69.14286	111.9679	207	0	103.5	119.5115	1474548	18062.4	60.9	115.195	381	2	609	101.5	177
11	88	879	9	4	656	3064	313	0	72.88889	136.1538	1532	0	766	884.5006	4232082	14789.53	73.25	209.0551	734	1	879	109.875	281
12	88	1160	9	6	3134	3048	1552	0	348.2222	682.4826	1518	0	508	782.3472	5329310	12931.03	82.85714	218.2461	836	1	1160	145	32
13	88	524	7	4	2812	2820	1397	0	401.7143	679.9149	1410	0	705	814.0639	10748092	20992.37	52.4	83.67822	284	1	524	87.33333	143
14	1034	6	1	1	6	6	6	6	6	0	6	6	6	0	2000000	333333.3	6	0	6	6	0	0	
15	88	1119	9	6	3160	3060	1565	0	351.1111	688.215	1524	0	510	785.4456	5558534	13404.83	79.92857	190.2907	704	1	1119	139.875	26
16	389	18378	13	13	4160	5724	1672	0	320	612.974	2634	0	440.3077	817.3089	537817	1414.735	735.12	3222.6	16177	1	18378	1531.5	462
17	88	822	7	4	458	356	220	0	65.42857	105.6265	178	0	89	102.7683	990267.6	13382	82.2	177.6662	579	1	822	137	269
18	88	876	9	4	630	2942	300	0	70	130.4224	1471	0	735.5	849.2822	4077626	14840.18	73	209.1993	734	1	876	109.5	281
19	88	1009	7	4	2748	2780	1365	0	392.5714	664.3006	1390	0	695	802.5169	5478692	10901.88	100.9	248.1028	804	1	1009	168.1667	340
20	88	616	7	4	484	414	233	0	69.14286	111.9679	207	0	103.5	119.5115	1457792	17857.14	61.6	119.0063	393	1	616	102.6667	180
21	1045	5	1	1	6	6	6	6	6	0	6	6	6	0	2400000	400000	5	0	5	5	0	0	
22	88	946	9	4	654	3064	312	0	72.66667	135.7129	1532	0	766	884.5006	3930233	13742.07	78.83333	193.2228	689	1	946	118.25	248
23	88	927	9	6	3160	3060	1565	0	351.1111	688.215	1524	0	510	785.4456	6709817	16181.23	66.21429	181.8356	694	2	927	115.875	277



	BE	BF	BG	BH	BI	BJ	BK	BL	BM	BN	BO	BP	BQ	BR	BS	BT	BU	BV	BW	BX	BY	BZ	CA
37786	0	0	0	0	0	0	2	12	0	0	253	-1	1	20	0	0	0	0	0	0	0	0	BENIGN
37787	0	0	0	0	0	0	50	14274	65	6392	254	4046	23	20	5268995	50832.37	5353839	5184849	9791595	15639.86	9806637	9758868	BENIGN
37788	0	0	0	0	0	0	15	19897	6	0	29200	362	10	32	0	0	0	0	0	0	0	0	BENIGN
37789	0	0	0	0	0	0	2	617	0	0	229	-1	0	32	0	0	0	0	0	0	0	0	BENIGN
37790	0	0	0	0	0	0	2	14	0	0	227	-1	0	32	0	0	0	0	0	0	0	0	BENIGN
37791	0	0	0	0	0	0	2	0	0	0	229	-1	0	32	0	0	0	0	0	0	0	0	BENIGN
37792	0	0	0	0	0	0	3	0	0	0	395	-1	0	32	0	0	0	0	0	0	0	0	BENIGN
37793	0	0	0	0	0	0	20	176	24	424	29200	227	12	32	0	0	0	0	0	0	0	0	BENIGN
37794	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37795	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37796	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37797	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37798	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37799	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37800	0	0	0	0	0	0	2	8	1	2	1024	29200	1	20	0	0	0	0	0	0	0	0	PortScan
37801	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37802	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37803	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37804	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37805	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37806	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37807	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37808	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37809	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37810	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan
37811	0	0	0	0	0	0	1	2	1	6	1024	0	0	24	0	0	0	0	0	0	0	0	PortScan

Label Encoding and Class Distribution

The CICIDS 2017 original dataset assigns string labels to each row, 'Benign' indicates normal traffic and various attack types indicating malicious flows. In this project, binary encoding was applied, Benign = 0, and all attack types = 1. This simplifies the drift prediction task because it reduces the label distribution to a single scaler value, that is the fraction of attack traffic in each batch, which then serves as the primary input to the KL, JS, and Wasserstein Divergence calculations.

Choosing Monday as a reference distribution

In the dataset the Monday traffic is entirely benign which creates a clean operational baseline that would exist at the operation of any real time production deployment. In a real intrusion detection system, a model is usually trained during a period of known good behaviour, and the aim is to detect when live traffic diverges from that baseline. Monday's 529,481 row batch serves as the known good training.

6. METHODOLOGY AND PIPELINE ARCHITECTURE

The pipeline comprises of four sequential phases where each phase operates on the outputs of the previous phase. All the outputs in between each phase are stored for reproducibility. The whole pipeline is operated using main.py, which calls everyone .py files in a definite sequence.

Phase 1: Preprocessing Pipeline

The first obstacle encountered during the development of this project was that the initial attempt to load all 8 CSV files simultaneously into a single panda's data frame caused memory error. The combined data set exceeds available RAM on a standard development machine when concatenated as a single object or a single CSV file.

The solution used was chunked streaming approach. The load_all_raw files () function reads each CSV in segments of 20,000 rows using pandas' chunk size parameter. Each chunk is labelled with a source_file column, preserving temporal batch identity throughout the entire pipeline. The complete preprocessing including the cleaning, feature engineering, and output writing processed all the 2,827,876 rows without a single memory error.

Step	
Low Variance features removed	10 features
Highly Correlated features dropped	21 features
Features left after selection	46 features
Preprocessed_data_v1.csv	2,827,876 rows
Dropped	2,867 rows

Data cleaning

The raw CICIDS 2017 files contain multiple well documented quality issues. Each column name carries leading and trailing white spaces, which causes silent lookup failures. To resolve this, we used str_strip() to all column names on file load.

Several numerical columns contain infinite values because of division by zero in flow rate calculations, which is highly observable in Wednesday's high volume DoS traffic. All infinite values were replaced by NaN and then removed with other missing values later. The preprocessing log confirms that 2,867 rows were removed across all eight files at this stage.

Feature engineering

Feature selection steps were applied on Monday's data to prevent information leakage from future temporal batches. If we fit features on the combined dataset this would allow Tuesday's and Wednesday's distribution to influence the features which may be considered as data leakage, as future test data is influencing baseline data, which is not desirable.

Low Variance Removal: SciKit-Learn's Variance Threshold was applied on Monday's feature set which led to removal of 10 features with zero variance across Monday's rows.

High Correlation Removal: The absolute Pearson Correlation Matrix was calculated based on Monday's data. One feature from each pair with correlation above 0.95 was dropped removing 21 more features.

After the performance of these two steps the dataset had 46 high valued features which were extracted from 77 columns this feature set was saved as processed_data_v1.csv and this is what is used in further pipeline for operator.

Phase 2: Baseline Model Training

Five different baseline classifiers were trained on 100,00 row sample derived from the processed dataset using 80/20 splitting mechanism to split the data set into training and testing sets. For standardization StandardScalar was fitted only on the training set and then applied consistently to the test split.

The five baseline models include:

```
models = {
    "random_forest": RandomForestClassifier(
        n_estimators=100,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        verbose=1,
    ),
    "xgboost": XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        verbosity=1,
    ),
    "logistic": LogisticRegression(
        max_iter=1000,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    "decision_tree": DecisionTreeClassifier(
        max_depth=10,
        min_samples_leaf=50,
        random_state=RANDOM_STATE,
    ),
    "svm": LinearSVC(
        C=1.0,
        max_iter=2000,
        random_state=RANDOM_STATE,
    ),
}
```

The outputs and artifacts from each model were saved for the reproducibility and documentation later.

Phase 3: Drift Competition and Temporal Simulation

Batch Splitting

The processed dataset was split into eight temporal batches using source_file column preserved during Phase 1. Split maintains strict chronological ordering without any randomization or shuffling; this preserves the temporal structure of original traffic data.

Statistical Divergence Metrics

For all the batches other than Monday, three divergence metrics were calculated relative to Monday's data reference at both the label level and the per feature level.

KL Divergence (current, reference): Computed on binary label distribution. A small epsilon of 1e-10 was added to all the probabilities to handle the case where Monday has zero attack traffic, otherwise it could produce log (0) error.

Jensen-Shannon Divergence: The symmetric bounded variant of KL. Computed using: 0.5*KL (current, M) + 0.5*KL (reference, M),

Where, M is mixture distribution.

JS divergence is always observed to be between 0 and 1 which makes it suitable for threshold-based alerting.

Wasserstein Distance (or Earth Mover's Distance): It measures the minimum cost of transforming the updated distribution into the reference training distribution. his metric was observed to be very informative in the rolling window analysis when detecting sharp overnight traffic transitions.

Fixed Reference and Rolling Window

In this project the drift was computed in two complementary modes. Drift compares every individual day's to distribution against Monday's fixed baseline, providing a cumulative view of how much the data has drifted from the original training distribution. Rolling window drift compares each day to the immediately previous day which captures sudden and sharp transitions that the fixed reference view might miss or smooth over.

Cross-Dataset Evaluation

This is a critical part of the project. A model trained only on Monday's 529,481 row batch and then evaluated without any retraining. The accuracies for each batch was recorded and these pairs (KL Divergence, Accuracy) forms the meta dataset for Phase 4. This automatically simulates the worst-case scenario, a model trained exclusively on one time and then deployed without any updates, the data distribution keeps shifting as time passes.

Phase 4: Drift Prediction Metamodel

The KL, JS, mean feature JS, and accuracy produced in phase 3 form a small meta-dataset. Regression models were trained on this meta-dataset.

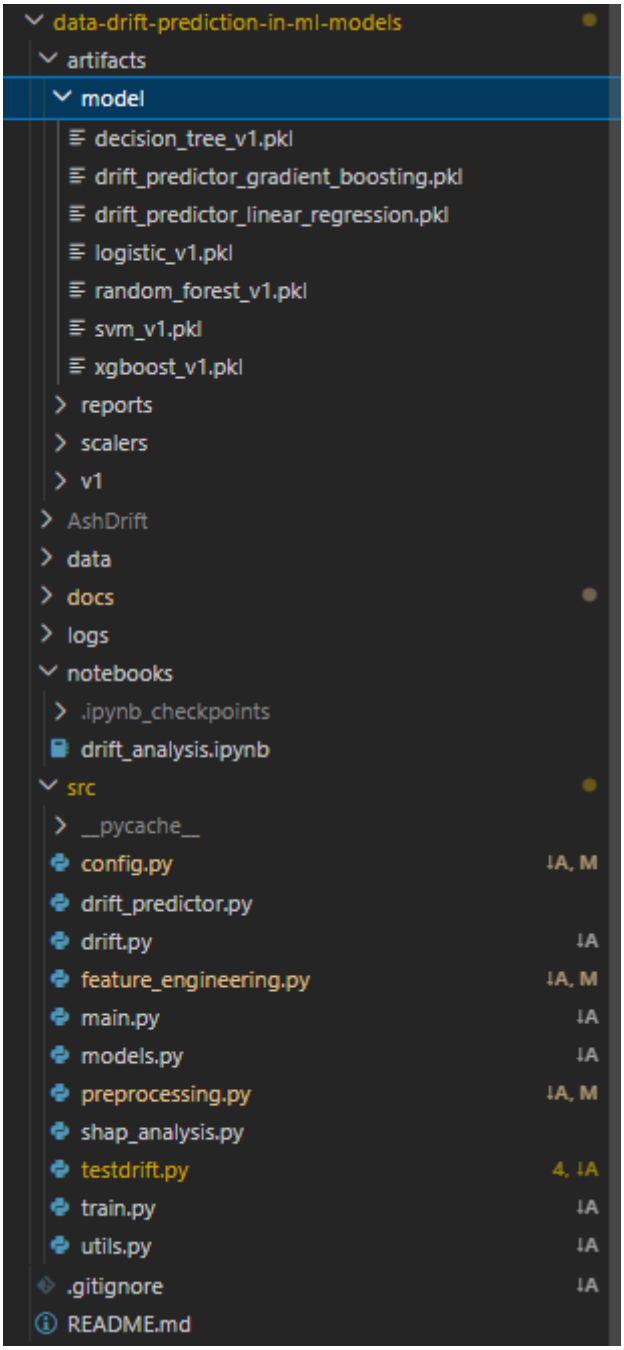
Linear Regression: Predicts cross data set accuracy from KL Divergence, JS Divergence, Mean Feature JS.

Gradient Boosting Regressor: 50 estimators with maximum depth of 2; same input features

Because of the small sample size observed from the 7 days, Leave-One-Out Cross-Validation (LOO-CV) was used as validation strategy. In Leave-One-Out Cross-Validation each sample is at least once held out as the test set, while the model is retrained on the remaining 6 samples.

This produces seven test predictions covering the entire dataset without any kind of data leakage.

```
models = {
    "linear_regression"       : LinearRegression(),
    "gradient_boosting"       : GradientBoostingRegressor(n_estimators=50, max_depth=2, random_state=42),
}
```

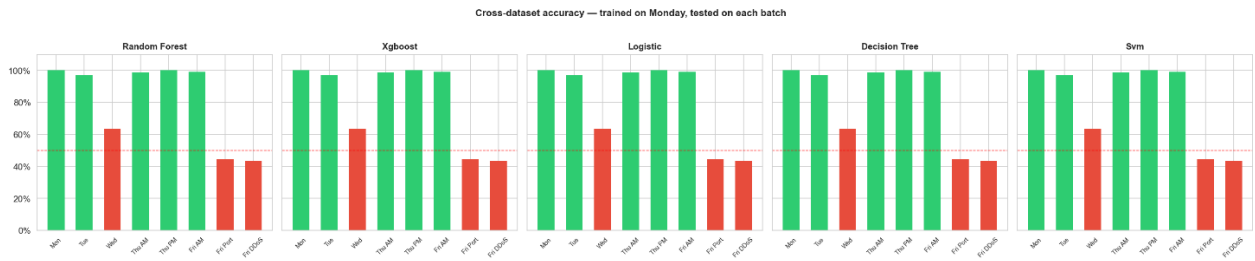


7. OBSERVATIONS AND RESULTS

7.1 Baseline Model Performance

All the five classifiers used were trained on 80,000 rows and tested on 20,000 rows derived from the processed dataset using 46 features and StandardScalar was applied to the training split exclusively.

Model	Accuracy	Precision	Recall	F1-Score	Scaled or not
Random Forest	99.76%	99.94%	99.84%	99.89%	No
XGBoost	99.79%	99.93%	99.81%	99.87%	No
Decision Tree	99.29%	~99.0%	~98.0%	~98.0%	No
Logistic Regression	93.83%	~86.0%	~84.0%	~85.0%	Yes
SVM (LinearSVC)	93.59%	0.96 / 0.83	0.96 / 0.85	0.96 / 0.84	Yes



Our dataset has non-linear nature which is better captured by the tree models. These outputs serve as the ceiling values and the cross-dataset evaluation shows how far below the ceiling performance lays when the data distribution is shifted overtime.

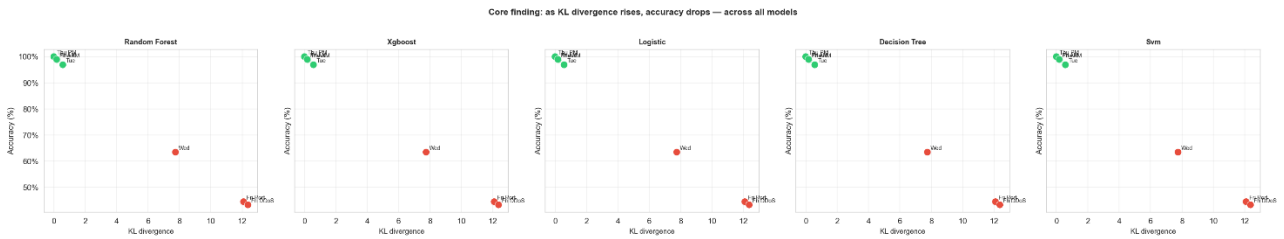
7.2 Label Distribution Across Temporal Batches

Batch	Benign (Label=0)	Attack (Label=1)
Monday (Reference)	100.00%	0.00%
Tuesday	96.89%	3.11%
Wednesday	63.48%	36.52%
Thursday Morning	98.82%	1.18%
Thursday Afternoon	99.99%	0.01%
Friday Morning	98.96%	1.04%
Friday PM (PortScan)	44.52%	55.48%
Friday PM (DDoS)	43.33%	56.67%

7.3 Cross Data Evaluation Result

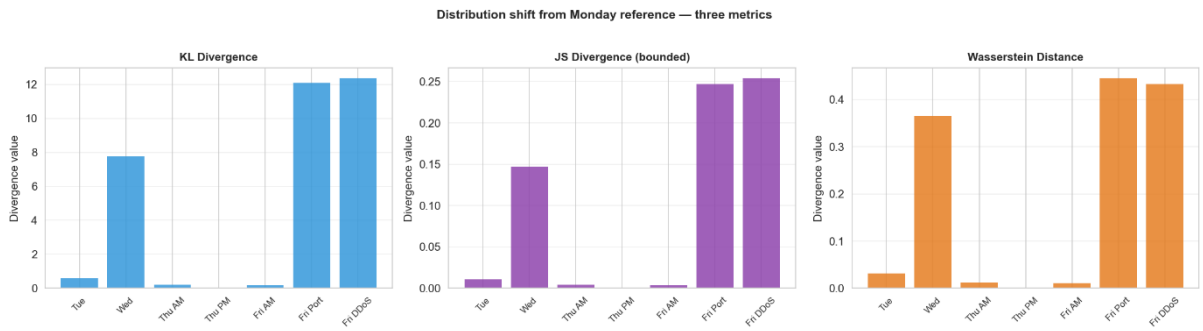
Batch	Cross-Dataset Accuracy	KL Divergence (Fixed)	Interpretation
Monday (Self)	100.00%	0.0000	Perfect: same distribution
Tuesday	96.89%	0.5770	Slight degradation; model robust
Wednesday	63.48%	7.7521	Significant collapse; first alert
Thursday AM	98.82%	0.2071	Recovery; low attack fraction

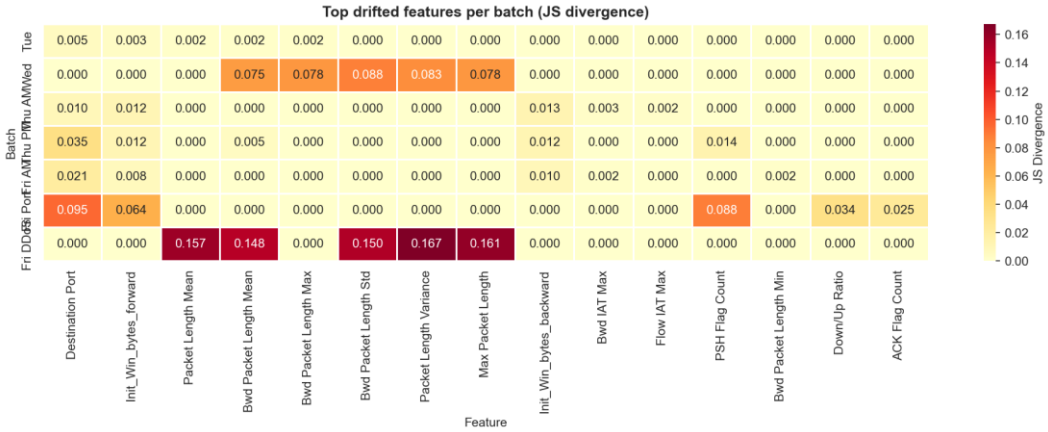
Thursday PM	99.99%	0.0018	Near perfect; essentially benign
Friday AM	98.96%	0.1821	Still robust; low scan volume
Friday PM (PortScan)	44.52%	12.0876	Severe collapse; below chance
Friday PM (DDoS)	43.33%	12.3652	Catastrophic failure; 43.33%



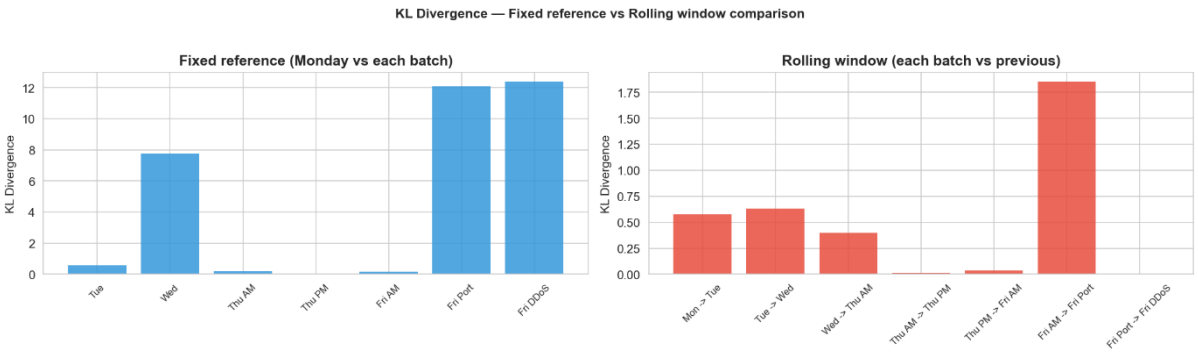
7.4 Fixed Reference Drift Score

Batch	KL Divergence	JS Divergence	Wasserstein Distance	Mean Feature JS
Tuesday (Brute Force)	0.5770	0.0109	58,459.05	0.00059
Wednesday (DoS/Hulk)	7.7521	0.1471	1,841,756.38	0.0235
Thursday AM (Web Attacks)	0.2071	0.0041	108,933.20	0.0013
Thursday PM (Infiltration)	0.0018	0.0000	178,856.05	0.0027
Friday AM (Port Scan)	0.1821	0.0036	114,944.30	0.0013
Friday PM (PortScan)	12.0876	0.2469	390,731.10	0.0110
Friday PM (DDoS)	12.3652	0.2540	984,674.10	0.0341





7.5 Rolling Window Drift Scores

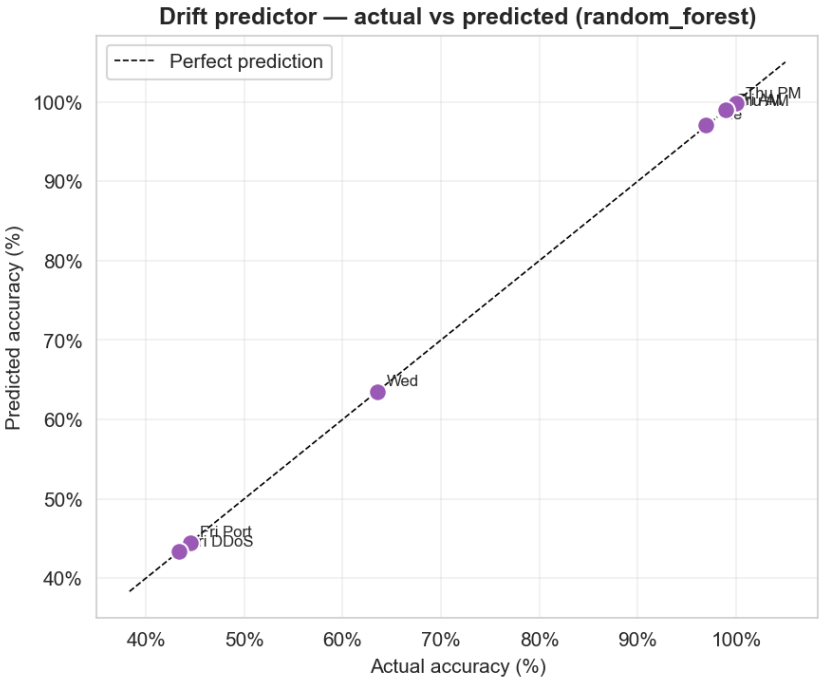


7.6 Drift Prediction Meta-Model Results

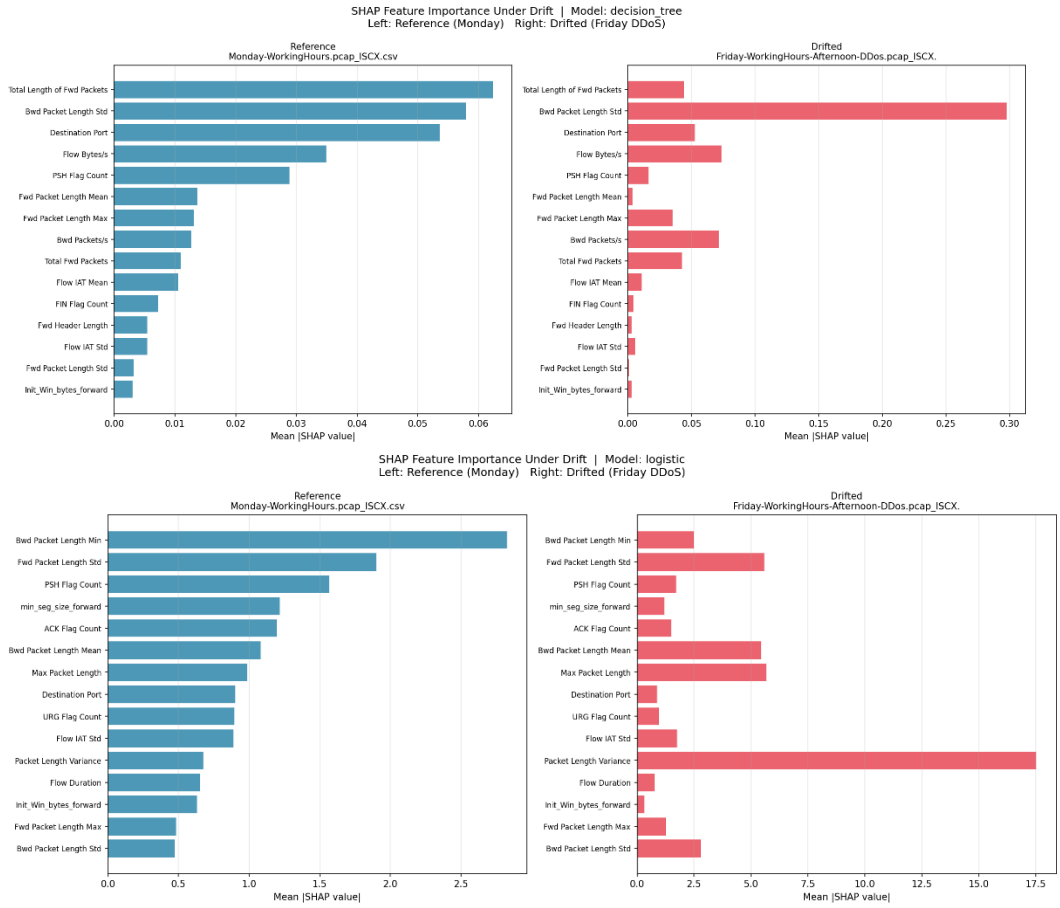

```
You, 2 days ago | 1 author (You)
[      You, 2 days ago • Add revised pipeline again
    {
        "batch": "Wednesday-workingHours.pcap_ISCX.csv",
        "average_accuracy": 0.6348,
        "threshold": 0.8,
        "per_model_accuracy": {
            "random_forest": 0.6348,
            "xgboost": 0.6348,
            "logistic": 0.6348,
            "decision_tree": 0.6348,
            "svm": 0.6348
        },
        "label_kl_divergence": 7.752125,
        "label_js_divergence": 0.147126,
        "predicted_accuracy": null,
        "alert": "DRIFT DETECTED \u2014 avg accuracy below threshold"
    },
    {
        "batch": "Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv",
        "average_accuracy": 0.4452,
        "threshold": 0.8,
        "per_model_accuracy": {
            "random_forest": 0.4452,
            "xgboost": 0.4452,
            "logistic": 0.4452,
            "decision_tree": 0.4452,
            "svm": 0.4452
        },
        "label_kl_divergence": 12.08758,
        "label_js_divergence": 0.246916,
        "predicted_accuracy": null,
        "alert": "DRIFT DETECTED \u2014 avg accuracy below threshold"
    },
    {
        "batch": "Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv",
        "average_accuracy": 0.4333,
        "threshold": 0.8,
        "per_model_accuracy": {
            "random_forest": 0.4333,
            "xgboost": 0.4333,
            "logistic": 0.4333,
            "decision_tree": 0.4333,
            "svm": 0.4333
        },
        "label_kl_divergence": 12.365228,
        "label_js_divergence": 0.253996,
        "predicted_accuracy": null,
        "alert": "DRIFT DETECTED \u2014 avg accuracy below threshold"
    }
]
```

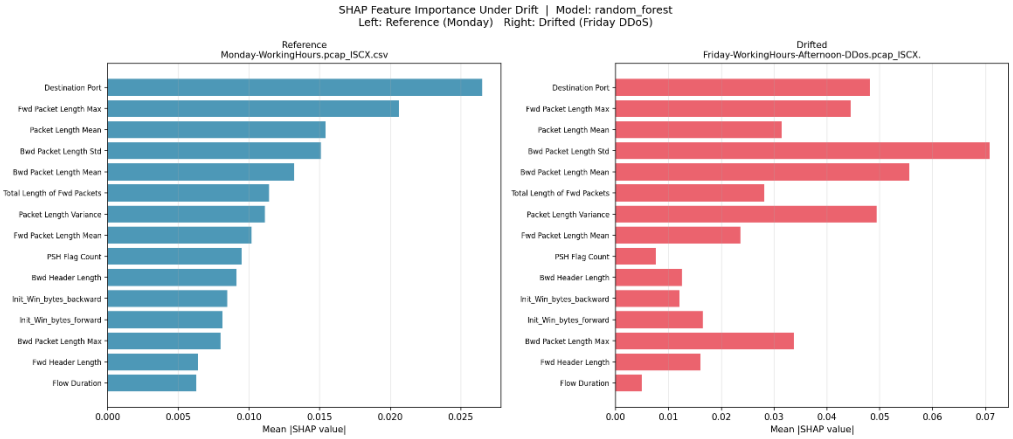

```
You, 2 days ago | 1 author (You)
Date: 2026-04-04 22:11
Model: drift_predictor_linear_regression
Task: Drift prediction (accuracy regression)
Input features: ['kl_divergence', 'js_divergence', 'mean_feature_js']
Samples: 7
CV method: Leave-one-out
LOO-CV MAE: 0.0143
LOO-CV R2: 0.9924
```

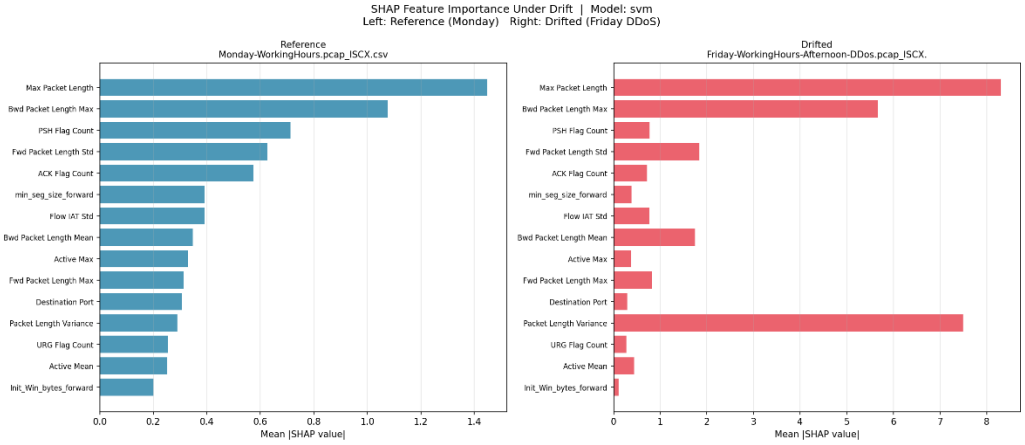
```
You, 2 days ago | 1 author (You)
Date: 2026-04-04 22:11
Model: drift_predictor_gradient_boosting
Task: Drift prediction (accuracy regression)
Input features: ['kl_divergence', 'js_divergence', 'mean_feature_js']
Samples: 7
CV method: Leave-one-out
LOO-CV MAE: 0.0694
LOO-CV R2: 0.8279
```

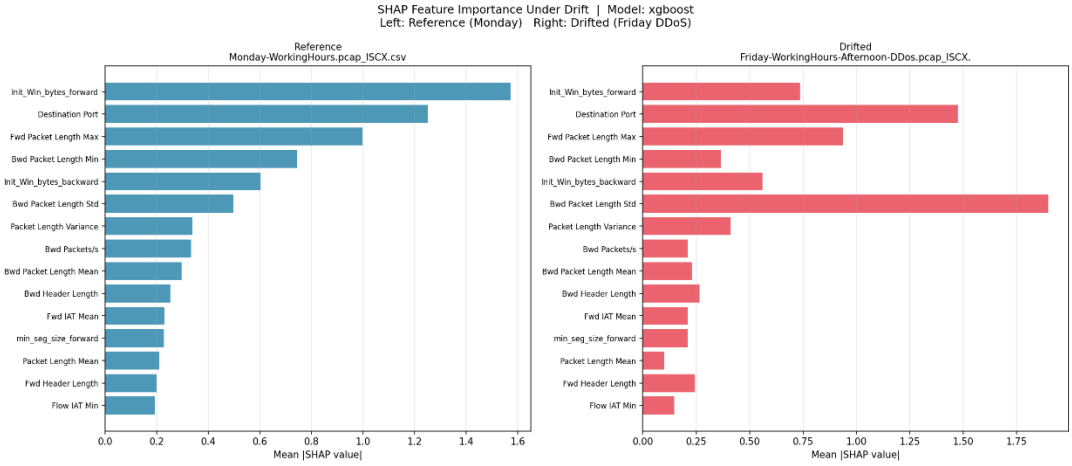


7.7 Feature-Level Drift Analysis (SHAP)




SHAP Feature Importance Under Drift | Model: random_forest
Left: Reference (Monday) Right: Drifted (Friday DDoS)
Reference
Monday-WorkingHours.pcap_ISCX.csv
Destination Port
Fwd Packet Length Max
Packet Length Mean
Bwd Packet Length Std
Bwd Packet Length Mean
Total Length of Fwd Packets
Packet Length Variance
Fwd Packet Length Mean
PSH Flag Count
Bwd Header Length
Init_Win_bytes_backward
Init_Win_bytes_forward
Bwd Packet Length Max
Fwd Header Length
Flow Duration
0.000
0.005
0.010
0.015
0.020
0.025
Mean |SHAP value|
Drifted
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.
Destination Port
Fwd Packet Length Max
Packet Length Mean
Bwd Packet Length Std
Bwd Packet Length Mean
Total Length of Fwd Packets
Packet Length Variance
Fwd Packet Length Mean
PSH Flag Count
Bwd Header Length
Init_Win_bytes_backward
Init_Win_bytes_forward
Bwd Packet Length Max
Fwd Header Length
Flow Duration
0.00
0.01
0.02
0.03
0.04
0.05
0.06
0.07
Mean |SHAP value|


SHAP Feature Importance Under Drift | Model: svm
Left: Reference (Monday) Right: Drifted (Friday DDoS)
Reference
Monday-WorkingHours.pcap_ISCX.csv
Max Packet Length
Bwd Packet Length Max
PSH Flag Count
Fwd Packet Length Std
ACK Flag Count
min_seg_size_forward
Flow IAT Std
Bwd Packet Length Mean
Active Max
Fwd Packet Length Max
Destination Port
Packet Length Variance
URG Flag Count
Active Mean
Init_Win_bytes_forward
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Mean |SHAP value|
Drifted
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.
Max Packet Length
Bwd Packet Length Max
PSH Flag Count
Fwd Packet Length Std
ACK Flag Count
min_seg_size_forward
Flow IAT Std
Bwd Packet Length Mean
Active Max
Fwd Packet Length Max
Destination Port
Packet Length Variance
URG Flag Count
Active Mean
Init_Win_bytes_forward
0
1
2
3
4
5
6
7
8
Mean |SHAP value|


SHAP Feature Importance Under Drift | Model: xgboost
Left: Reference (Monday) Right: Drifted (Friday DDoS)
Reference
Monday-WorkingHours.pcap_ISCX.csv
Init_Win_bytes_forward
Destination Port
Fwd Packet Length Max
Bwd Packet Length Min
Init_Win_bytes_backward
Bwd Packet Length Std
Packet Length Variance
Bwd Packets/s
Bwd Packet Length Mean
Bwd Header Length
Fwd IAT Mean
min_seg_size_forward
Packet Length Mean
Fwd Header Length
Flow IAT Min
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
Mean |SHAP value|
Drifted
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.
Init_Win_bytes_forward
Destination Port
Fwd Packet Length Max
Bwd Packet Length Min
Init_Win_bytes_backward
Bwd Packet Length Std
Packet Length Variance
Bwd Packets/s
Bwd Packet Length Mean
Bwd Header Length
Fwd IAT Mean
min_seg_size_forward
Packet Length Mean
Fwd Header Length
Flow IAT Min
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
Mean |SHAP value|

8. CONCLUSION

The proposed method was run on 2,827,876 live network flows in eight different windows, five base models with accuracies more than 93% during evaluation, distance metrics calculated between every pair of datasets, and a meta-regression model created to predict cross-dataset accuracy scores through an R-squared of 0.9924 and MAE of 1.43%. In this instance, the most crucial observation is that when Monday's Random Forest classifier encounters Friday's DDoS attack traffic dataset, the classifier becomes totally inefficient due to its accuracy dropping from 100% to merely 43.33%, which is predictable via KL divergence score of 12.3652.

GitHub: https://github.com/Anwesha-code/data-drift-prediction-in-ml-models

References

[1] Sharafaldin, Iman, Arash Habibi Lashkari, and Ali A. Ghorbani. "A detailed analysis of the cicids2017 data set." *International conference on information systems security and privacy*. Cham: Springer International Publishing, 2018.

[2] Panigrahi, Ranjit, and Samarjeet Borah. "A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems." *International Journal of Engineering & Technology* 7.3.24 (2018): 479-482.

[3] Dataset: https://www.kaggle.com/datasets/sateeshkumar6289/cicids-2017-dataset

[4] Bifet, Albert, and Ricard Gavalda. "Learning from time-changing data with adaptive windowing." *Proceedings of the 2007 SIAM international conference on data mining*. Society for Industrial and Applied Mathematics, 2007.

[5] Helfstein, Lucas, and Kelly Rosa Braghetto. "An Empirical Analysis of Data Drift Detection Techniques in Machine Learning Systems." *Simpósio Brasileiro de Banco de Dados (SBBD)*. SBC, 2024.